

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

4. Giao tiếp Đồng bộ — REST, gRPC & Resilience	4
4.1. Giao tiếp đồng bộ — Khi nào nên, khi nào tránh?	5
4.1.1. Request-Response: mô hình quen thuộc	5
4.1.2. Ưu và nhược điểm	5
4.1.3. Design-time Coupling vs Runtime Coupling	6
4.1.4. Khi nào dùng sync?	7
4.1.5. Interaction Styles — Phân loại kiểu tương tác	7
4.1.6. REST vs gRPC — Hai lựa chọn cho sync	8
4.2. OpenFeign — Declarative REST Client	9
4.2.1. Vấn đề: gọi service khác không đơn giản như gọi hàm	9
4.2.2. OpenFeign: gọi REST như gọi hàm	9
4.2.3. Cách hoạt động	10
4.3. Service Discovery & Load Balancing	11
4.3.1. Vấn đề: URL cứng trong distributed system	11
4.3.2. Service Discovery patterns	11
4.3.3. Netflix Eureka trong KBM	12
4.3.4. Resilience Metrics — Đo lường độ tin cậy	13
4.4. Resilience Patterns — Sống sót trong Distributed System	14
4.4.1. Tại sao cần resilience?	14
4.4.2. Bốn resilience patterns cốt lõi	15
4.4.3. Triển khai thực tế với Resilience4j — Code ví dụ cho LMS	17
4.5. Case Study: KBM	20
4.5.1. Bài toán	20
4.5.2. Hiện trạng: Strategy Pattern + OpenFeign	21
4.5.3. Phân tích các vấn đề	21
4.5.4. Đề xuất migration	22
4.5.5. Tích hợp ngoài: QLĐT, GitHub Classroom và Network Practice	23
4.6. Tổng kết	24
4.7. Đọc thêm	25
Tài liệu tham khảo	25

4.

CHƯƠNG 4.

Giao tiếp Đồng bộ – REST, gRPC & Resilience

“Microservices favor smart endpoints and dumb pipes. The communication infrastructure should be simple; intelligence belongs in the services.”

— Sam Newman, *Building Microservices* [4a]

MỤC TIÊU HỌC TẬP

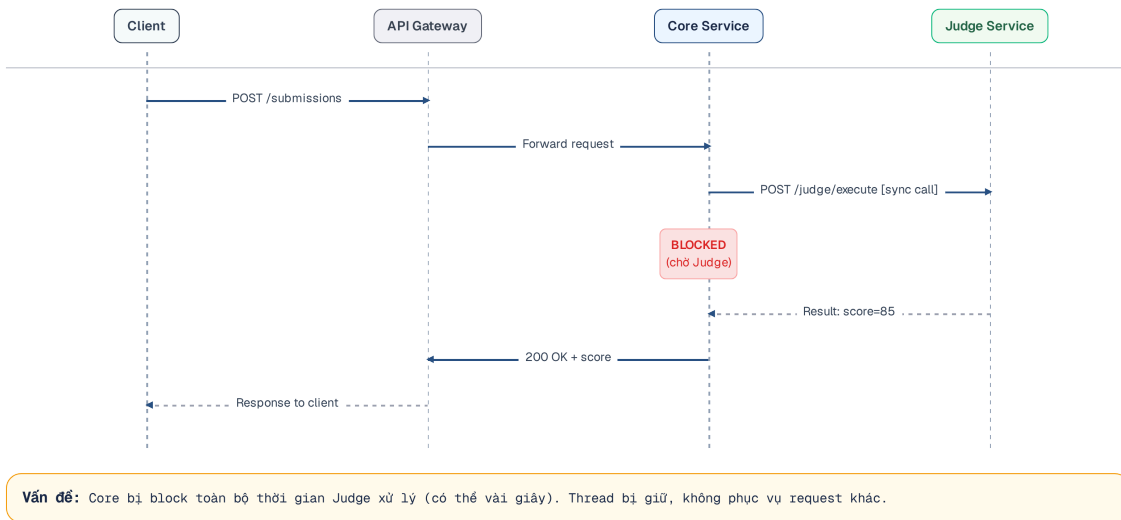
- Hiểu các kiểu tương tác (interaction styles) trong microservices
- So sánh REST và gRPC — khi nào dùng cái nào
- Sử dụng OpenFeign để gọi service-to-service một cách declarative
- Hiểu Service Discovery và Load Balancing với Eureka
- Áp dụng các resilience patterns: Circuit Breaker, Retry, Timeout, Bulkhead
- Phân tích bài toán dispatch SQL và tích hợp ngoài trong KBM

4.1. Giao tiếp đồng bộ — Khi nào nên, khi nào tránh?

Trước khi phân tích chi tiết, hãy hình dung giao tiếp đồng bộ giống như việc sinh viên lên thẳng Phòng Đào tạo để nộp đơn và phải đứng đợi cho đến khi chuyên viên xử lý xong mới được ra về. Sự chờ đợi này đảm bảo tính chắc chắn, nhưng lại gây tắc nghẽn khi có quá đông người.

4.1.1. Request-Response: mô hình quen thuộc

Giao tiếp đồng bộ (*synchronous communication*) là mô hình tương tác cơ bản và quen thuộc nhất đối với hầu hết các nhà phát triển phần mềm. Trong mô hình này, khi service A cần thông tin hoặc yêu cầu xử lý từ service B, nó sẽ gửi một request qua mạng và bị chặn (block) và phải **chờ đợi** cho đến khi nhận được response trả về. Chỉ khi có kết quả từ service B, service A mới tiếp tục thực thi các logic nghiệp vụ tiếp theo của nó. HTTP và REST hiện đang là giao thức phổ biến nhất để hiện thực hóa mô hình này nhờ tính đơn giản, dễ tích hợp và khả năng khai thác hệ sinh thái công cụ phong phú. Tuy nhiên, đằng sau sự đơn giản đó là những chi phí ngầm về độ trễ và sự phụ thuộc chặt chẽ về mặt thời gian giữa các thành phần hệ thống.



Hình 4.1: Giao tiếp đồng bộ — Core Service blocked trong khi chờ Judge response

4.1.2. Ưu và nhược điểm

Bảng dưới đây tổng hợp các ưu và nhược điểm cốt lõi của giao tiếp đồng bộ:

	Ưu điểm	Nhược điểm
Đơn giản	Mô hình request-response quen thuộc, dễ gỡ lỗi (debug)	Bên gọi (caller) bị chặn (block) hoặc đình trệ cho đến khi nhận được phản hồi (response)
Nhất quán	Biết ngay kết quả (thành công/thất bại)	Temporal coupling : cả hai service phải online cùng lúc
Tooling	HTTP tooling phong phú (Postman, curl, browser)	Cascading failures : service B ngừng hoạt động → A cũng ngừng hoạt động
Tracing	Request ID dễ dàng được theo dõi (trace) xuyên suốt chuỗi các lời gọi dịch vụ	Latency accumulation : $A \rightarrow B \rightarrow C = \text{tổng latency}$

Bảng 4.1: Ưu và nhược điểm của giao tiếp đồng bộ

Richardson phân tích rõ trong [2a, Ch.3]: giao tiếp đồng bộ tạo **temporal coupling** (cả hai service phải sẵn sàng cùng lúc) và **runtime dependency** (service gọi không thể hoạt động nếu service được gọi ngừng hoạt động). Đây là lý do kiến trúc microservices thường ưu tiên giao tiếp bất đồng bộ (async) cho các luồng xử lý không yêu cầu phản hồi tức thời.

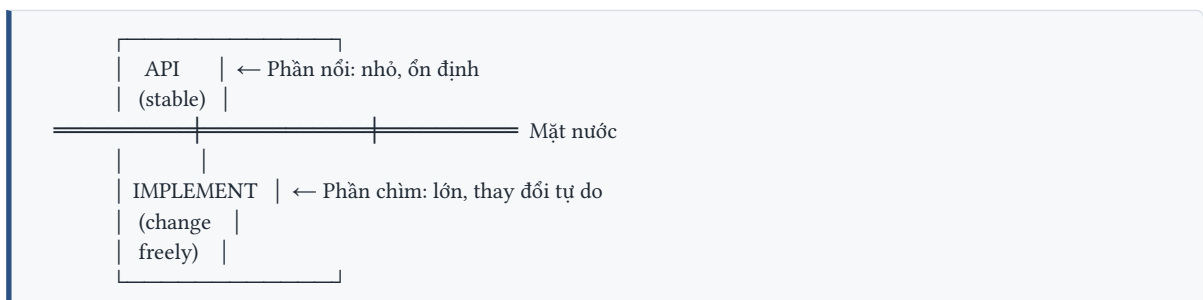
4.1.3. Design-time Coupling vs Runtime Coupling

Richardson trong phiên bản thứ hai [2b, Ch.4] phân biệt rõ hai loại coupling cơ bản — hiểu sai sự khác biệt này là nguyên nhân phổ biến nhất dẫn đến thiết kế microservices kém:

	Design-time Coupling	Runtime Coupling
Định nghĩa	Thay đổi service A → phải thay đổi service B	Service A cần B đang chạy để xử lý request
Khi nào xảy ra	Khi phát triển, compile, deploy	Khi hệ thống đang chạy (runtime)
Ảnh hưởng	Giảm team autonomy, tăng coordination	Giảm availability, tăng latency
Ví dụ KBM	Core Service đổi DTO format → Auth Service phải cập nhật parser	Core Service gọi sync Judge → Judge ngừng hoạt động → Core trả lỗi
Giải pháp	API versioning (§3.2), backward compatible changes	Async messaging (Ch.5), Circuit Breaker (§4.4)

Bảng 4.2: Hai loại coupling trong microservices

Iceberg Principle — Richardson gọi đây là nguyên tắc nền tảng cho loose coupling [2b, Ch.4]:



API (phần nổi) phải nhỏ nhất có thể — chỉ bộc lộ (expose) những gì thành phần gọi/sử dụng dịch vụ (consumer) thực sự cần. Implementation (phần chìm) có thể lớn và thay đổi tự do mà không làm ảnh hưởng đến các consumer này. **Đây cũng là lý do dùng DTO pattern (§3.4)** thay vì trả entity trực tiếp — entity là “phần chìm” không nên lộ ra.

🔗 Nghịch lý DRY và Coupling

Trong monolith, DRY (Don't Repeat Yourself) gần như luôn đúng. Nhưng trong microservices, DRY có thể tạo coupling không mong muốn. Ví dụ: Lớp `OrderTotalCalculator` được sử dụng chung thông qua một thư viện chia sẻ (shared library). Khi đó, mỗi lần cập nhật quy tắc nghiệp vụ (business rule), tất cả các dịch vụ phụ thuộc vào thư viện này đều phải được nâng cấp đồng loạt (lock-step deployment). Đôi khi **duplication có chủ đích** tốt hơn coupling — đặc biệt khi logic có thể phân kỳ hợp lệ giữa các contexts. Richardson trong [2b, Ch.4] khuyên: chỉ áp dụng nguyên tắc DRY khi các triển khai phân kỳ (divergent implementations) thực sự gây ra lỗi (**bugs**), không nên áp dụng một cách khiên cưỡng trong mọi trường hợp.

4.1.4. Khi nào dùng sync?

Giao tiếp đồng bộ phù hợp khi:

Cần phản hồi ngay – query data (GET), validation, authentication

Flow đơn giản – chuỗi gọi dịch vụ ngắn ($A \rightarrow B$), không phải chuỗi dài ($A \rightarrow B \rightarrow C \rightarrow D$)

Các thao tác nghiêng về truy xuất dữ liệu (Read-heavy operations) – lấy thông tin, không phải thay đổi trạng thái phức tạp

Không phù hợp khi:

Các tác vụ thực thi kéo dài (Long-running operations) – chấm bài SQL mất 5–30 giây

Tác vụ Gửi-và-quên (Fire-and-forget) – gửi thông báo (notification), ghi nhận nhật ký hệ thống (log event)

Giao dịch liên dịch vụ (Cross-service transactions) – cần saga pattern (Chương 6)

🔗 Quy tắc ngón tay cái

Nếu bên gọi (caller) *phải biết* kết quả để tiếp tục xử lý → chọn giao tiếp đồng bộ. Nếu bên gọi chỉ cần *kích hoạt* (trigger) hành động và không cần đợi kết quả → chọn giao tiếp bất đồng bộ (Chương 5). Richardson trong [2a, Ch.3] minh họa rõ: sử dụng giao tiếp đồng bộ để *validate* (vì cần kết quả ngay lập tức), nhưng sử dụng giao tiếp bất đồng bộ để *process* (vì không cần chờ đợi). Trong hệ thống KBM, việc truy vấn trạng thái hoạt động (health query) của Judge sử dụng cơ chế đồng bộ, nhưng việc gửi yêu cầu chấm bài (submit) lại sử dụng cơ chế bất đồng bộ thông qua Kafka — phản ánh hai nhu cầu hoàn toàn khác biệt.

4.1.5. Interaction Styles — Phân loại kiểu tương tác

Richardson phân loại các kiểu giao tiếp trong microservices dựa trên hai không gian chính: số lượng bên tiếp nhận (one-to-one so với one-to-many) và tính đồng bộ (synchronous so với asynchronous) [2a, Ch.3]. Trong khi giao tiếp đồng bộ thường bị giới hạn ở kiểu một-kèm-một (như Request/Response quen thuộc), thì giao tiếp bất đồng bộ cung cấp tính linh hoạt cao hơn. Nó không chỉ hỗ trợ gọi một-kèm-một (như thông báo một chiều hay request/response bất đồng bộ), mà còn đặc biệt hiệu quả ở mô hình một-nhiều

(như Publish/Subscribe hay Publish/Async responses) nơi một thông điệp có thể kích hoạt hàng loạt dịch vụ cùng lúc.

Hầu hết service dùng kết hợp nhiều kiểu. Ví dụ: KBM Core Service dùng *request/response* (Feign) để query Judge, *publish/subscribe* (Kafka) để gửi submissions, và *one-way notification* (WebSocket) để push kết quả.

4.1.6. REST vs gRPC — Hai lựa chọn cho sync

Khi quyết định sử dụng giao tiếp đồng bộ, REST (dựa trên HTTP và JSON) thường là lựa chọn mặc định và phổ biến nhất trong hệ sinh thái phần mềm. Tuy nhiên, REST không phải là công cụ duy nhất. **gRPC** — một framework RPC (Remote Procedure Call) mã nguồn mở hiệu năng cao do Google phát triển — đã nổi lên như một lựa chọn thay thế phổ biến thứ hai. Công nghệ này đặc biệt phù hợp trong các giao tiếp nội bộ giữa các microservices đòi hỏi tốc độ cao, độ trễ thấp và băng thông được tối ưu hóa bằng định dạng nhị phân. Bảng sau đây sẽ so sánh chi tiết giữa hai công nghệ này để giúp quá trình lựa chọn giao thức kiến trúc được chính xác và sát với nhu cầu thực tế hơn.

Tiêu chí	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Format	JSON (text, human-readable)	Protocol Buffers (binary, compact)
Performance	Chậm hơn (text parsing)	Thường nhanh hơn đáng kể, nhất là với khối lượng dữ liệu (payload) lớn / cần nhiều vòng lặp giao tiếp (round-trip) (binary + HTTP/2 multiplexing); mức cải thiện tùy thuộc vào tải hệ thống (workload)
Contract	OpenAPI spec (optional)	<code>.proto</code> file (bắt buộc)
Code gen	Optional (OpenAPI generators)	Built-in (protoc generates client/server)
Browser support	Native	Cần gRPC-Web proxy
Streaming	Không native (phải dùng SSE/WebSocket)	Bi-directional streaming native
Debugging	Dễ dàng (hỗ trợ curl, Postman, trình duyệt)	Phức tạp (cần gRPC tools)

Bảng 4.3: REST vs gRPC — so sánh chi tiết

gRPC Architecture — Tại sao nhanh hơn? gRPC nhanh hơn REST không chỉ nhờ binary encoding. Ba yếu tố kiến trúc quan trọng:

1. **HTTP/2 multiplexing** — Nhiều requests cùng dùng một TCP connection (không cần mở connection mới mỗi request). Quan trọng khi service A gọi service B hàng trăm lần/giây.
2. **Protocol Buffers** — Schema-first, binary encoding. Payload nhỏ hơn JSON 3-5x. Schema evolution built-in (backward/forward compatible) — `.proto` file là contract.
3. **Four communication patterns**: Unary (request-response, như REST), Server streaming (server gửi stream), Client streaming (client gửi stream), **Bi-directional streaming** (cả hai gửi stream đồng thời). **Lưu ý quan trọng**: Bi-directional streaming rất hiệu quả cho backend-to-backend. Trình duyệt (thông qua gRPC-Web) có hỗ trợ Server-Streaming, nhưng chưa hỗ trợ Bi-directional streaming. Dù có thể dùng Server-streaming để trả kết quả (ví dụ: stream kết quả từng test case cho trình duyệt

của sinh viên), WebSockets hoặc SSE (Server-Sent Events) vẫn là chuẩn phổ biến hơn cho real-time frontend.

4. **Lưu ý về Load Balancing:** Vì gRPC duy trì kết nối HTTP/2 dài hạn (persistent connection), các Load Balancer L4 thông thường (phân phối theo kết nối TCP) sẽ chia tải bị lệch. Để load balancing gRPC hiệu quả, bắt buộc phải dùng L7 Load Balancer (như Envoy, HAProxy) hoặc Service Mesh để load balancing trên từng request.

Khi nào dùng gRPC thay REST?

Giao tiếp nội bộ (Internal service-to-service) – Khi hiệu năng là ưu tiên hàng đầu và không yêu cầu tương tác trực tiếp từ trình duyệt → chọn gRPC

API công khai / Giao diện người dùng (Frontend) – Khi cần sự hỗ trợ từ trình duyệt và dễ dàng gỡ lỗi thủ công → chọn REST

Truyền phát dữ liệu (Streaming) – Khi yêu cầu giao tiếp hai chiều thời gian thực (real-time bi-directional) → chọn gRPC (hoặc WebSocket)

Môi trường Polyglot – đội ngũ phát triển sử dụng nhiều ngôn ngữ lập trình khác nhau → gRPC (code gen đa ngôn ngữ)

KBM không còn là hệ thống REST thuần

LMS core của KBM vẫn ưu tiên REST/JSON vì browser, dashboard và OpenAPI tooling cần tính dễ debug. Nhưng Network Lab chủ động dùng SOAP/REST/gRPC/JNP để sinh viên thấy được sự khác biệt giữa các giao thức (protocol) trong cùng một lĩnh vực (domain) bài tập. Vì vậy, migration sang gRPC không phải mục tiêu “đổi toàn bộ REST”; cách đúng hơn là **chọn protocol theo boundary**: Sử dụng REST cho các API hướng người dùng (client-facing), gRPC cho các giao tiếp nội bộ đòi hỏi thông lượng cao hoặc truyền phát dữ liệu (high-throughput/streaming) khi cần thiết, và các hàng đợi lô/công việc (batch/job queue) cho các tác vụ chấm điểm (judge) kéo dài hoặc mang tính trạng thái (stateful).

4.2. OpenFeign — Declarative REST Client

Thay vì bắt sinh viên tự cầm hồ sơ gõ cửa từng phòng ban, hệ thống một cửa OpenFeign tự động chuẩn hóa mọi thủ tục giao tiếp. Cách tiếp cận này giúp việc gọi dịch vụ nội bộ trở nên minh bạch và an toàn hơn đáng kể.

4.2.1. Vấn đề: gọi service khác không đơn giản như gọi hàm

Trong monolith, gọi module khác là một dòng code: `judgeService.execute(request)`. Trong microservices, cùng logic đó trở thành: xây dựng HTTP request, serialize payload, gán headers (JWT token), gửi qua network, xử lý response codes, deserialize result, handle timeout/error. Và điều nguy hiểm nhất: khi gọi hàm nội bộ, nếu có lỗi, chương trình sẽ ném ra ngoại lệ (throw Exception) ngay lập tức. Nhưng khi gọi qua mạng, hệ thống có thể bị đình trệ vô thời hạn mà không phát sinh ngoại lệ báo lỗi. Trong hệ thống phân tán, **một phản hồi chậm (slow response) còn nguy hiểm hơn rất nhiều so với một lỗi trả về ngay lập tức (fail-fast)**.

4.2.2. OpenFeign: gọi REST như gọi hàm

Để giải quyết sự cồng kềnh của các HTTP client truyền thống, hệ sinh thái Spring cung cấp **OpenFeign** (Spring Cloud OpenFeign). Công cụ này mang đến một cách tiếp cận mang tính khai báo (declarative), cho phép các lập trình viên chỉ cần định nghĩa các lời gọi API dưới dạng một Java interface thông thường.

Toàn bộ các tác vụ kỹ thuật phức tạp như thiết lập kết nối, quản lý header, và chuyển đổi dữ liệu sẽ được framework sinh ra mã thực thi (implementation) một cách tự động ngay tại runtime.

```
// #sym.times Manual REST call – verbose, boilerplate
RestTemplate restTemplate = new RestTemplate();
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(jwtToken);
HttpEntity<JudgeRequest> entity = new HttpEntity<>(request, headers);
ResponseEntity<JudgeResult> response = restTemplate.exchange(
    url, HttpMethod.POST, entity, JudgeResult.class
);

// #sym.checkmark Declarative Feign client – clean, type-safe
@FeignClient(name = "kbn-judge-sql")
public interface JudgeClient {
    @PostMapping("/api/judge/sql")
    JudgeResult submitSql(@RequestBody JudgeRequest request); // Gọi như hàm bình thường!
}
```

Listing 4.1: So sánh RestTemplate (verbose) và OpenFeign (declarative)

Việc chuyển đổi từ cách viết mã thủ công sử dụng `RestTemplate` sang mô hình khai báo của OpenFeign không chỉ giúp mã nguồn trở nên gọn gàng, sạch sẽ mà còn giảm thiểu đáng kể các lỗi tiềm ẩn do cấu hình thiếu sót. Tuy nhiên, sự trừu tượng hóa này cũng có thể vô tình che khuất một số thiết lập cấu hình mạng quan trọng ở tầng dưới, đặc biệt là cách quản lý vòng đời của các kết nối TCP mà nếu không chú ý sẽ nhanh chóng trở thành nút thắt cổ chai về hiệu năng của toàn hệ thống.

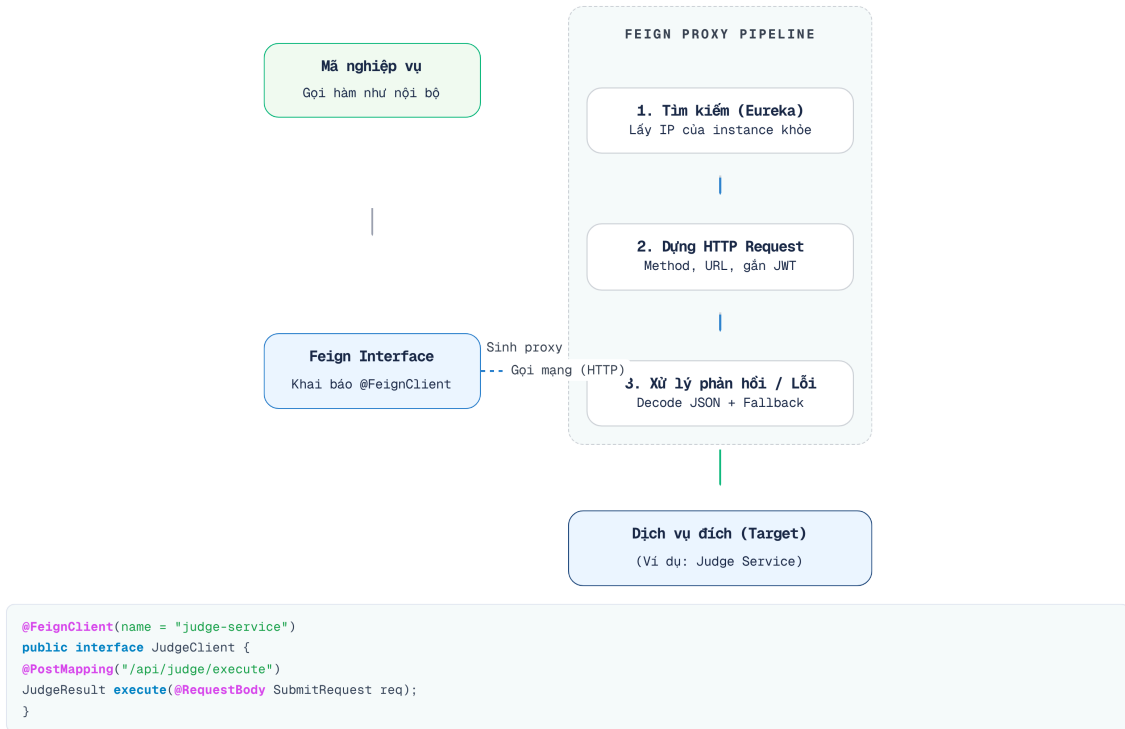
🔗 Cấu hình Connection Pooling cho OpenFeign

Mặc định, OpenFeign sử dụng `HttpURLConnection` của Java, không hỗ trợ connection pooling (mỗi yêu cầu tự mở và đóng một kết nối TCP riêng biệt, gây suy giảm hiệu năng nghiêm trọng do độ trễ thiết lập kết nối). Trong môi trường thực tế (production), team **bắt buộc** phải cấu hình OpenFeign sử dụng `Apache HttpClient` hoặc `OkHttp` để khai thác cơ chế quản lý connection pool, giúp giảm thiểu độ trễ mạng.

Việc duy trì một connection pool sẵn sàng cũng giống như việc trường học duy trì một đội ngũ tư vấn viên túc trực liên tục thay vì mỗi khi sinh viên cần hỏi mới bắt đầu gọi điện điều người từ nơi khác đến. Nhờ vậy, OpenFeign có thể tối ưu hóa luồng gọi dịch vụ bên dưới một cách trong suốt.

4.2.3. Cách hoạt động

Để hình dung rõ hơn sự hỗ trợ của công cụ này, hãy quan sát sơ đồ dưới đây minh họa các bước tự động hóa mà Feign thực hiện thay cho lập trình viên.



Hình 4.2: Cách Feign proxy tự động xử lý HTTP request, JWT, load balancing

Feign tự động xử lý: serialization/deserialization (JSON ↔ Java), header injection (JWT token qua `RequestInterceptor`), service discovery (Eureka lookup thay vì URL hardcoded), và error decoding (response code → exception). Giống như một chuyên viên một cửa, nó nhận yêu cầu từ chúng ta và tự động hoàn thiện mọi giấy tờ cần thiết để gửi lên phòng ban cấp trên.

4.3. Service Discovery & Load Balancing

Khi quy mô hệ thống trường học mở rộng, việc ghi nhớ vị trí chính xác của từng phòng ban trở nên bất khả thi. Service Discovery hoạt động như một bản đồ điện tử động của toàn trường, giúp các ứng dụng luôn tìm thấy nhau mà không cần cấu hình cứng.

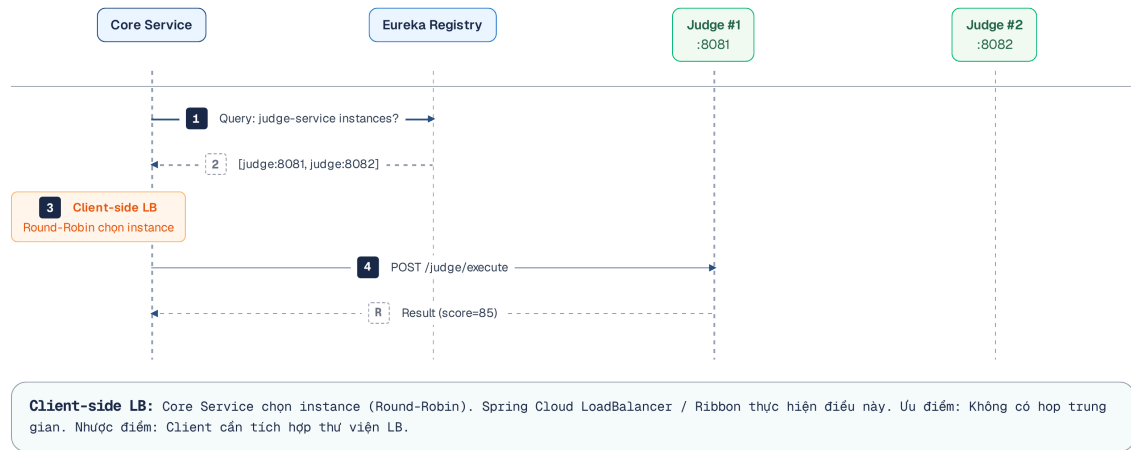
4.3.1. Vấn đề: URL cứng trong distributed system

Trong monolith, gọi module khác là gọi hàm — không cần biết “module ở đâu”. Trong microservices, mỗi service là một process riêng, chạy trên host:port khác nhau. Câu hỏi: **service A tìm service B ở đâu?** Cách đơn giản nhất: gán cứng (hardcode) URL trong cấu hình. Nhưng khi dịch vụ mở rộng quy mô (scale) thành 3 bản thể (instances), hệ thống nên sử dụng URL nào? Khi dịch vụ được triển khai (deploy) sang vùng chứa mới, địa chỉ IP thay đổi — cơ chế nào sẽ chịu trách nhiệm cập nhật?

4.3.2. Service Discovery patterns

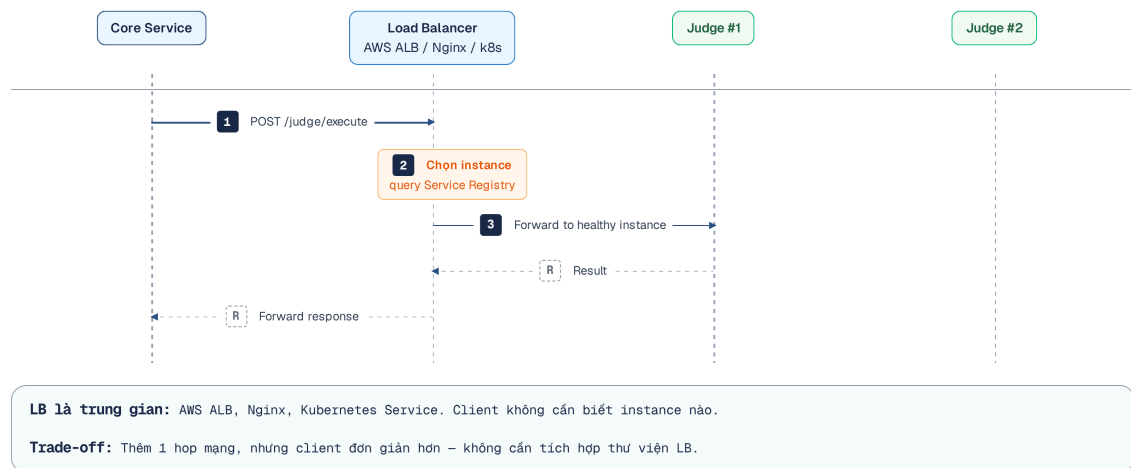
Có hai pattern chính [2a, Ch.3]:

Client-side discovery — Client (service gọi) tự query service registry để tìm danh sách instances, rồi tự load balance:



Hình 4.3: Client-side discovery — service tự query Eureka và load balance

Server-side discovery — Load balancer (hoặc API Gateway) hoạt động như một thành phần trung gian, client chỉ cần biết URL của load balancer:



Hình 4.4: Server-side discovery — load balancer điều hướng request

4.3.3. Netflix Eureka trong KBM

KBM sử dụng **Netflix Eureka** cho cụm Java/Spring services — client-side discovery pattern:

Khi microservice khởi động, nó đăng ký tại Eureka server với tên (`spring.application.name`) và địa chỉ. Gateway và các service khác query Eureka để tìm instances. Trong config Gateway: `uri: lb://kbm-core` nghĩa là tra cứu Eureka tìm tất cả instances có tên `kbm-core`, rồi phân tải (load balance) lưu lượng giữa các bản thể này.

❗ Service naming không nhất quán

Trong snapshot cũ của KBM, `kbm-core` và `kbm-academic` từng dùng `spring.application.name = "app"` — vi phạm nguyên tắc unique service identity [2a]. Khi Eureka nhận hai services cùng tên, nó coi chúng là instances của *cùng một service* → routing sai. Đây là hậu quả của việc Assignment tách ra từ Core nhưng không đổi service name. **Migration path:** đổi thành `kbm-core` và `kbm-academic`, cập nhật Eureka config và Gateway routes.

❗ Eureka vs Service Discovery của Kubernetes

Eureka là một *application-level* service discovery — service tự đăng ký và tự tra cứu trong code (qua Spring Cloud). Nếu hệ thống được triển khai trên **Kubernetes**, phần lớn vai trò này đã có sẵn ở tầng hạ tầng: mỗi service là một **Service** object, K8s lo đăng ký, DNS nội bộ (`http://kbm-core`) và load balancing — thường *không cần* Eureka nữa. Nói gọn: dùng Eureka khi chạy ngoài K8s (VM, bare-metal, Spring Cloud thuần); cân nhắc bỏ Eureka khi đã lên K8s để tránh trùng lặp hai lớp discovery. KBM hiện dùng Eureka vì cụm Java/Spring chưa chạy hoàn toàn trên K8s; lộ trình lên k3s (Chương 12) sẽ mở ra lựa chọn chuyển sang discovery K8s-native.

4.3.4. Resilience Metrics — Đo lường độ tin cậy

Để đánh giá độ tin cậy của một hệ thống, giới kỹ thuật thường sử dụng bốn thước đo cốt lõi. Đầu tiên là **MTBF (Mean Time Between Failures)**, chỉ số thể hiện thời gian trung bình giữa hai sự cố liên tiếp (ví dụ: Judge Service ngừng hoạt động 1 lần/tuần thì MTBF là 168 giờ). Khi sự cố xảy ra, tốc độ khắc phục được đo bằng **MTTR (Mean Time To Recovery)** — khoảng thời gian từ lúc hệ thống ngưng trệ đến khi khôi phục hoàn toàn. Khoảng thời gian hệ thống hoạt động trơn tru sau khi phục hồi cho đến sự cố tiếp theo được gọi là **MTTF (Mean Time To Failure)**. Cuối cùng, tổng hòa của các yếu tố này sẽ cho ra **Availability (Tính sẵn sàng)**, được tính bằng tỷ lệ phần trăm thời gian hệ thống thực sự phục vụ người dùng ($MTTF / (MTTF + MTTR)$).

Thuật ngữ “nines” (các số 9) thường được sử dụng trong ngành công nghiệp phần mềm để mô tả nhanh các mức độ khả dụng mục tiêu của một hệ thống. Mỗi “số 9” được thêm vào đại diện cho những thách thức kỹ thuật lớn, đi kèm với sự phức tạp và chi phí vận hành cao hơn. 4.4 minh họa quy đổi từ các tỷ lệ phần trăm này sang thời gian ngừng hoạt động tối đa cho phép, giúp chúng ta hình dung rõ ràng hơn về mức độ khắt khe của từng cấp độ cam kết dịch vụ.

Target	Downtime/năm	Downtime/tháng	Phù hợp cho
99% (two nines)	3.65 ngày	7.3 giờ	Internal tools, dev environments
99.9% (three nines)	8.76 giờ	43.8 phút	KBM production (phù hợp)
99.99% (four nines)	52.6 phút	4.38 phút	E-commerce, banking
99.999% (five nines)	5.26 phút	26.3 giây	Critical infrastructure

Bảng 4.4: Availability “nines” — downtime cho phép theo target

Những mục tiêu tính sẵn sàng này không chỉ là các con số lý thuyết, mà chúng thực sự định hình các quyết định thiết kế và cách vận hành hệ thống hàng ngày. Để biến những tỷ lệ phần trăm này thành một cơ chế quản lý rủi ro và điều tiết tốc độ phát hành phần mềm một cách thực tiễn, giới kỹ thuật áp dụng một nguyên tắc điều hành quan trọng là hạn mức lỗi.

Error Budgets

Google SRE (Site Reliability Engineering) đề xuất: nếu target availability = 99.9%, hệ thống có **error budget** = 0.1% downtime/tháng (43.8 phút). Ngân sách lỗi này sẽ được dùng để đánh đổi cho các hoạt động như: triển khai phiên bản mới (deployments), thử nghiệm tính năng (experimentation) hoặc bảo trì có kiểm soát (controlled maintenance). Khi cạn kiệt ngân sách lỗi, đội ngũ vận hành sẽ đóng băng việc triển khai (freeze deployments) và phải tập trung hoàn toàn vào việc duy trì tính ổn định của hệ thống. Đây là cách biến “availability target” từ khái niệm mơ hồ thành **nguyên tắc vận hành cụ thể** (xem thêm SLI/SLO/SLA ở Ch.11).

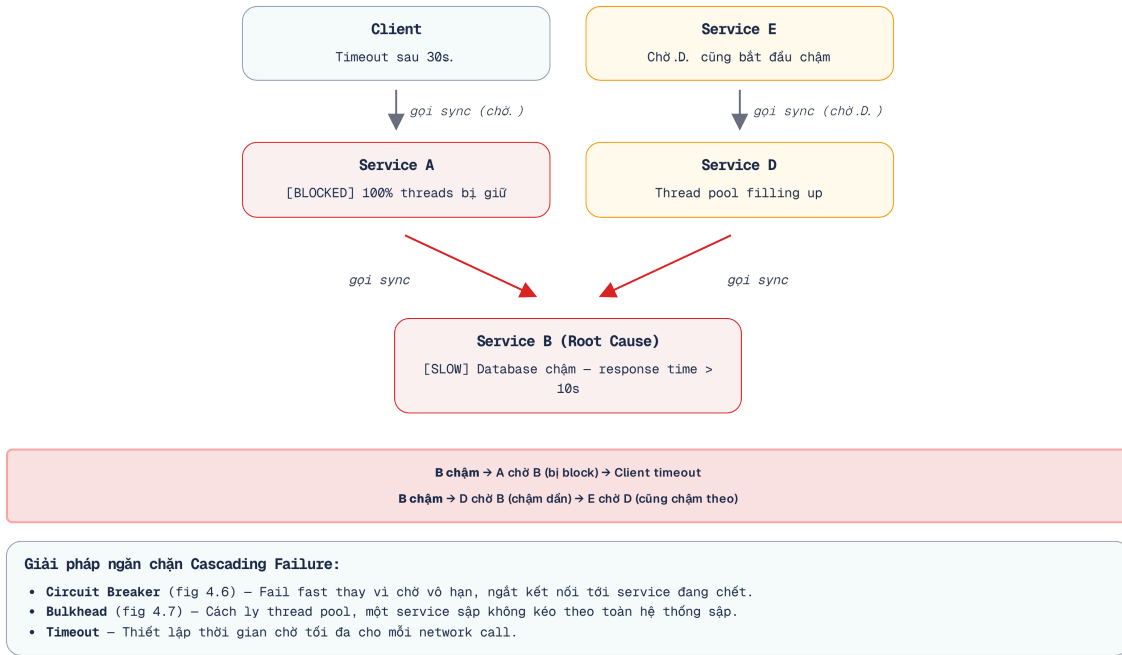
4.4. Resilience Patterns — Sống sót trong Distributed System

Môi trường mạng không đáng tin cậy cũng giống như việc mất điện hay rớt mạng cục bộ trong ký túc xá. Để hệ thống KBM không sụp đổ dây chuyền, chúng ta cần trang bị các lá chắn bảo vệ tự động, đảm bảo tiến trình học tập của sinh viên luôn được duy trì.

4.4.1. Tại sao cần resilience?

Trong monolith, nếu cơ sở dữ liệu chậm, *toàn bộ* ứng dụng chậm — nhưng ít nhất lỗi rõ ràng. Trong microservices, khi service B chậm, service A *vẫn chạy* nhưng các luồng xử lý (threads) bị chặn chờ B → các yêu cầu (requests) gửi đến A cũng bị chậm lại → service C gọi A cũng bị chậm → **cascading failure** lan khắp hệ thống [5, Ch.7]. Hiện tượng này được gọi là **Thread Pool Exhaustion** (Cạn kiệt luồng). Hãy tưởng tượng Web Server (như Tomcat) là một Phòng Giáo vụ chỉ có giới hạn 200 chuyên viên (200 worker threads). Nếu một dịch vụ phía sau (service B) bị chậm, toàn bộ 200 chuyên viên này sẽ phải đứng chờ, dẫn đến việc các sinh viên mới (request mới) bước vào Phòng Giáo vụ sẽ không có ai phục vụ, khiến hệ thống tê liệt hoàn toàn.

Hugo Rocha trong [5] nhấn mạnh: trong distributed system, *lỗi không phải ngoại lệ — lỗi là trạng thái bình thường*. Network sẽ timeout, services sẽ crash, databases sẽ chậm. Câu hỏi không phải “nếu lỗi xảy ra” mà là “khi lỗi xảy ra”.



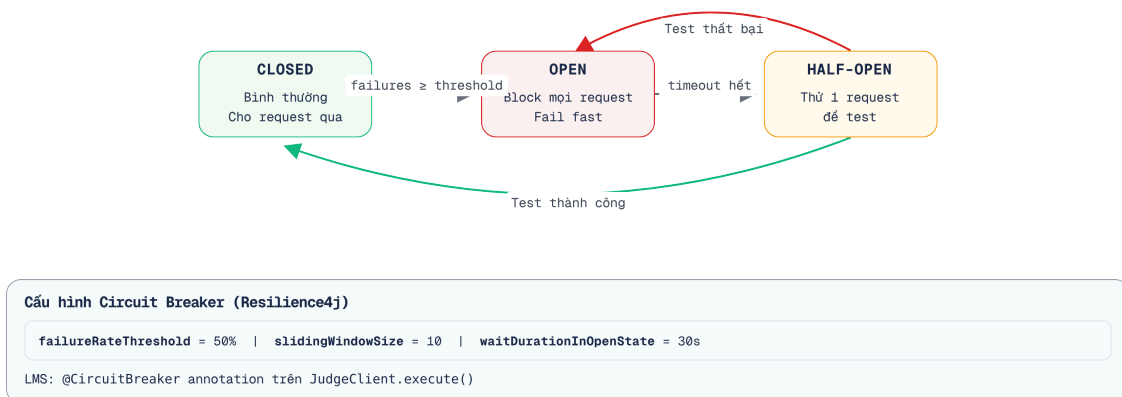
Hình 4.5: Cascading failure — Service B chậm làm A, D, E đều bị ảnh hưởng

4.4.2. Bốn resilience patterns cốt lõi

Để ngăn chặn lỗi dây chuyền (cascading failure), hệ thống phân tán áp dụng bốn cơ chế bảo vệ (resilience patterns) kinh điển. Lớp bảo vệ đầu tiên là **Timeout**, giúp cắt đứt những kết nối quá hạn để giải phóng tài nguyên. Đối với những lỗi mạng chập chờn mang tính tạm thời, pattern **Retry** sẽ thử lại kết nối. Khi dịch vụ phía sau gặp lỗi liên tục, **Circuit Breaker** sẽ ngắt mạch để chống quá tải. Cuối cùng, **Bulkhead** giam các rủi ro cạn kiệt tài nguyên vào những khu vực (thread pool) biệt lập, ngăn chặn lỗi lan rộng.

4.4.2.1. Circuit Breaker — Chi tiết state machine

Trọng tâm của cơ chế Circuit Breaker là một mô hình trạng thái (state machine), liên tục quan sát và chuyển đổi để quyết định có nên cho phép một yêu cầu mạng được gửi đi hay không. Thay vì để các lỗi xảy ra một cách thụ động, bộ ngắt mạch sẽ chủ động kiểm soát lưu lượng để bảo vệ các dịch vụ đang suy yếu khỏi những áp lực không cần thiết.



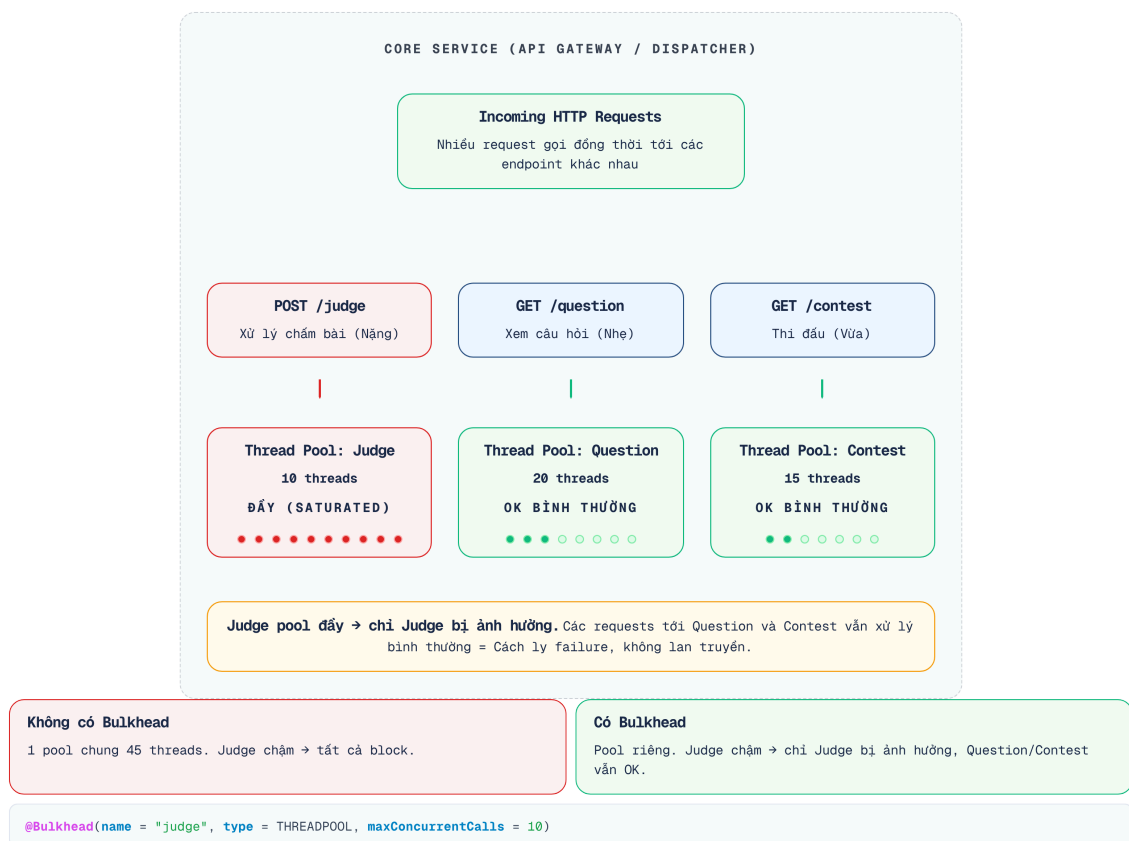
Hình 4.6: Circuit Breaker state machine — Closed, Open, Half-Open

Khi mạch ở trạng thái **Open**, mọi yêu cầu bị từ chối (reject) *ngay lập tức* và không được gửi tới dịch vụ đích (downstream). Cơ chế này bảo vệ cả hệ thống gọi (caller) khỏi tình trạng bị chặn luồng (thread blocking), và hệ thống được gọi (callee) khỏi sự quá tải (overwhelm) trong quá trình phục hồi (recovery).

Nhưng làm sao biết khi nào hệ thống đã phục hồi? Đó là lúc trạng thái **Half-Open** (mở một phần) phát huy tác dụng. Sau một khoảng thời gian chờ, Circuit Breaker sẽ chuyển sang Half-Open và cho phép một số lượng request giới hạn (ví dụ: 3 request) lọt qua để “thử nghiệm lưu lượng”. Nếu các request này thành công, Circuit Breaker tự động đóng lại (Closed) và hệ thống hoạt động bình thường. Nếu thất bại, nó lập tức ngắt lại (Open) để tiếp tục chờ đợi.

4.4.2.2. Bulkhead — Cách ly resources

Cơ chế tiếp theo trong hệ thống bảo vệ là Bulkhead, chuyên tập trung vào việc cô lập phạm vi thiệt hại khi một dịch vụ trong hệ thống bắt đầu bộc lộ sự chậm trễ bất thường. Nó thực hiện việc cô lập này bằng cách phân chia tài nguyên hệ thống — cụ thể là cấp phát các thread pool hoặc giới hạn quota (số lượng request đồng thời) riêng biệt cho từng nhóm nghiệp vụ hoặc từng API gọi ra ngoài. Thay vì sử dụng chung một pool luồng khổng lồ cho mọi request, Bulkhead đảm bảo rằng nếu một dịch vụ phản hồi chậm và chiếm dụng toàn bộ các luồng xử lý (hiện tượng Thread pool exhaustion), thì rủi ro cục bộ này sẽ chỉ bị khoanh vùng trong chính thread pool được cấp cho nó. Các khoang tài nguyên khác vẫn an toàn và trống trải, đảm bảo hệ thống không bị tê liệt toàn cục và các chức năng khác vẫn tiếp tục được phục vụ bình thường.



Hình 4.7: Bulkhead pattern — thread pool cách ly theo function

Tên “Bulkhead” lấy từ thiết kế tàu thủy: khoang tàu bị nước tràn vào, các khoang khác vẫn khô — tàu không chìm. Michael Nygard giới thiệu pattern này trong *Release It!* (2007) [5, Ch.7].

4.4.2.3. Lưu ý quan trọng cho Retry

Chỉ thử lại (retry) các thao tác có Idempotent (GET, PUT, DELETE). Việc thử lại các thao tác POST có thể tạo ra dữ liệu trùng lặp (duplicate data). Richardson trong [2a, Ch.3] nhấn mạnh: retry token (idempotency key) là bắt buộc khi retry non-idempotent operations.

4.4.2.4. Kết hợp các patterns

Trong thực tế, các patterns thường kết hợp theo thứ tự:

Request → Retry (3 lần, backoff) → Circuit Breaker → Timeout (5s) → Bulkhead → Actual Call

Chuỗi kết hợp trên tạo ra một quy trình kiểm soát lỗi đa tầng. Khi một yêu cầu mạng được gửi đi, nó phải vượt qua từng chốt kiểm tra này trước khi thực sự chạm đến hệ thống đích. Mặc dù cấu trúc tuần tự này trông rất trực quan trên giấy, nhưng khi triển khai thực tế thông qua các thư viện, thứ tự bọc (wrapping) của các cơ chế này có thể gây ra nhiều lỗi logic.

i Resilience4j execution order

Retry phải là lớp ngoài cùng — nếu Timeout bao Retry, một timeout đầu tiên sẽ huỷ toàn bộ retry chain, vô hiệu hoá mục đích của Retry. Thứ tự chính xác theo tài liệu Resilience4j:

Retry > CircuitBreaker > TimeLimiter > Bulkhead .

Spring Cloud Circuit Breaker + Resilience4j là stack phổ biến nhất cho Java microservices. Chỉ cần thêm dependency và annotation — không cần viết logic retry/circuit breaker thủ công.

4.4.3. Triển khai thực tế với Resilience4j — Code ví dụ cho LMS

Để minh họa cách áp dụng các nguyên lý bảo vệ này vào thực tế, chúng ta sẽ xem xét phần triển khai cụ thể tại dịch vụ cốt lõi của hệ thống KBM. Spring Cloud Circuit Breaker kết hợp cùng Resilience4j cung cấp một bộ công cụ cho phép tích hợp cả bốn cơ chế bảo vệ thông qua các annotation. Cách tiếp cận này tách biệt logic nghiệp vụ khỏi logic xử lý lỗi và quản lý độ tin cậy.

```
// Core Service — gọi Judge với đầy đủ resilience patterns
@Service
public class JudgeCallService {
    private final JudgeClient judgeClient;

    // Thủ tục khai báo annotation không ảnh hưởng đến luồng thực thi (execution) — Spring AOP sử dụng @Order của aspect
    // Thủ tục thực thi thực tế: Retry → CircuitBreaker → TimeLimiter → Bulkhead → call
    @Retry(name = "judge", fallbackMethod = "judgeFallback")
    @CircuitBreaker(name = "judge")
    @TimeLimiter(name = "judge")
    @Bulkhead(name = "judge")
    public CompletableFuture<JudgeResult> executeSQL(JudgeRequest request) {
        return CompletableFuture.supplyAsync() ->
            judgeClient.submitSql(request)
        );
    }

    // Phương thức dự phòng (Fallback) khi mạch bị ngắt (OPEN) hoặc vượt quá số lần thử lại (retries)
    private CompletableFuture<JudgeResult> judgeFallback(
        JudgeRequest request, Throwable t) {
        log.warn("Judge unavailable: {}, submissionId={}",
            t.getMessage(), request.getSubmissionId());
        return CompletableFuture.completedFuture(
            JudgeResult.pending("Sandbox đang bận, bài sẽ được chấm khi sẵn sàng")
        );
    }
}
```

Listing 4.2: Resilience4j annotations cho Judge Feign Client

Đoạn mã trên minh họa cách phối hợp nhiều lớp bảo vệ trên cùng một phương thức khai báo, tạo thành một cơ chế phòng vệ trước các rủi ro sụp đổ dây chuyền. Tuy nhiên, việc đẩy các tác vụ gọi mạng chậm chạp vào một tiến trình xử lý bất đồng bộ thông qua các đối tượng `CompletableFuture` cũng mang lại những rủi ro tiềm ẩn về việc cạn kiệt tài nguyên luồng của hệ thống, đặc biệt là nếu người lập trình không thực sự nắm vững cơ chế phân bổ thread pool mặc định đang diễn ra bên dưới.

⚠ Cấp Executor cho CompletableFuture

Đoạn `supplyAsync(...)` ở trên dùng overload *một tham số*, nên task chạy trên **common ForkJoinPool** dùng chung toàn JVM. Khi kết hợp với `@TimeLimiter` và `@Bulkhead`, các lệnh gọi kéo dài (chờ phản hồi từ Judge) có thể làm cạn kiệt bể luồng dùng chung (thread starvation), gây đình trệ toàn bộ các tác vụ khác trong hệ thống. Trong môi trường thực tế (production), nên truyền một **Executor** riêng (`supplyAsync() -> ..., judgeExecutor`) hoặc dùng `@Bulkhead(type = Bulkhead.Type.THREADPOOL)` để Resilience4j quản lý thread pool tách biệt cho nhóm call này.

Bên cạnh việc thiết lập qua các annotation trực tiếp trên mã nguồn, hiệu năng thực sự và sự linh hoạt của Resilience4j nằm ở khả năng tinh chỉnh cấu hình thông qua các tệp đặc tả bên ngoài. Điều này mang lại lợi thế quản trị: cho phép dễ dàng thay đổi các ngưỡng cảnh báo và bảo vệ ngay trong quá trình hệ thống đang chạy (runtime) mà không cần biên dịch và triển khai lại toàn bộ dự án.

```
# application.yml — Resilience4j configuration
resilience4j:
  circuitbreaker:
    instances:
      judge:
        slidingWindowSize: 10      # Đánh giá trên 10 requests gần nhất
        failureRateThreshold: 50   # Mở circuit khi 50% requests lỗi
        waitDurationInOpenState: 30s # Chờ 30s trước khi thử lại (half-open)
        permittedNumberOfCallsInHalfOpenState: 3 # Thử 3 calls khi half-open
        recordExceptions:          # Chỉ count những exceptions này
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - feign.FeignException.ServiceUnavailable

  retry:
    instances:
      judge:
        maxAttempts: 3             # Tối đa 3 lần thử
        waitDuration: 1s           # Chờ 1s giữa các lần
        enableExponentialBackoff: true # Kích hoạt backoff
        exponentialBackoffMultiplier: 2 # 1s → 2s → 4s (exponential)
        retryExceptions:
          - java.io.IOException      # Chỉ retry lỗi tạm thời
          - java.util.concurrent.TimeoutException # Bắt buộc để Retry bắt được timeout từ TimeLimiter
        ignoreExceptions:
          - com.lms.exception.BusinessException # KHÔNG retry lỗi logic

  timelimiter:
    instances:
      judge:
        timeoutDuration: 5s        # Timeout 5s cho mỗi call

  bulkhead:
    instances:
      judge:
        maxConcurrentCalls: 10     # Tối đa 10 calls đồng thời đến Judge
        maxWaitDuration: 500ms     # Chờ tối đa 500ms nếu bulkhead đầy
```

Listing 4.3: Resilience4j YAML configuration cho LMS

Các cấu hình YAML phía trên đã thiết lập các ngưỡng an toàn cho quá trình tương tác giữa hệ thống Core và hệ thống chấm điểm Judge. Tuy nhiên, những tham số này hiếm khi là những giá trị tĩnh có thể áp dụng chung chung cho mọi quy mô dự án phần mềm. Tùy thuộc vào đặc điểm hoạt động của từng môi trường và mức độ tải dự kiến của hệ thống, các kỹ sư cần chủ động giám sát và tinh chỉnh các thông số này để đạt được điểm cân bằng: vừa đủ nhạy để ngắt mạch khi có lỗi thật sự, vừa đủ linh hoạt để không gây ngắt quãng dịch vụ khi mạng chỉ chậm chèn tạm thời. 4.5 dưới đây cung cấp một số giá trị tham khảo cốt lõi được khuyến nghị cho các kịch bản triển khai khác nhau.

Parameter	Development	Production (LMS)	High-traffic
<code>slidingWindowSize</code>	5	10	20
<code>failureRateThreshold</code>	50%	50%	30%
<code>waitDurationInOpenState</code>	10s	30s	60s
<code>retry.maxAttempts</code>	1	3	3
<code>timelimiter.timeoutDuration</code>	10s	5s	3s
<code>bulkhead.maxConcurrentCalls</code>	5	10	25

Bảng 4.5: Giá trị recommend cho từng environment

Việc thiết lập các thông số này chỉ là một phần của quá trình cấu hình Resilience4j. Một yếu tố khác quan trọng không kém, và thường bị bỏ qua dẫn đến các lỗi khó phát hiện, chính là thứ tự áp dụng các annotation decorator trên cùng một phương thức.

💡 Thứ tự áp dụng annotations

Theo tài liệu Resilience4j, các decorator được lồng theo thứ tự mặc định **từ ngoài vào trong**: `Retry( CircuitBreaker( TimeLimiter( Bulkhead( call )))`. Nghĩa là Retry nằm ở lớp ngoài cùng — mỗi lần retry sẽ chạy lại toàn bộ chuỗi `CircuitBreaker` → `TimeLimiter` → `Bulkhead`. Thứ tự này quan trọng: nếu `TimeLimiter` bao bọc bên ngoài Retry, một ngoại lệ quá hạn (timeout) đầu tiên sẽ hủy bỏ toàn bộ chuỗi thử lại (retry chain), làm vô hiệu hóa hoàn toàn mục đích của cơ chế Retry. Cấu hình mặc định của Resilience4j đã xử lý đúng thứ tự này [5, Ch.7]. Hơn nữa, vì `TimeLimiter` ném ra `TimeoutException` khi hết hạn, ta **bắt buộc phải khai báo** `java.util.concurrent.TimeoutException` trong danh sách `retryExceptions` của Retry. Nếu thiếu, khi timeout xảy ra, Retry sẽ không kích hoạt và báo lỗi ngay lập tức.

💬 Design for Failure

“In distributed systems, failure is not an exception — it’s a normal state. Design every inter-service call assuming it will fail. Timeout, Retry, Circuit Breaker, Bulkhead — these are not optional, they are prerequisites.”

— Tổng hợp từ Hugo Rocha [5, Ch.7] và Michael Nygard, *Release It!*

4.5. Case Study: KBM

Để hiểu rõ sự phối hợp giữa các thành phần, hãy xem xét hệ thống chấm điểm SQL của KBM như một hội đồng thi. Trong đó, Coordinator hoạt động như thư ký hội đồng điều phối đề thi, còn các DBMS Workers là các giảng viên chuyên trách từng môn học khác nhau.

4.5.1. Bài toán

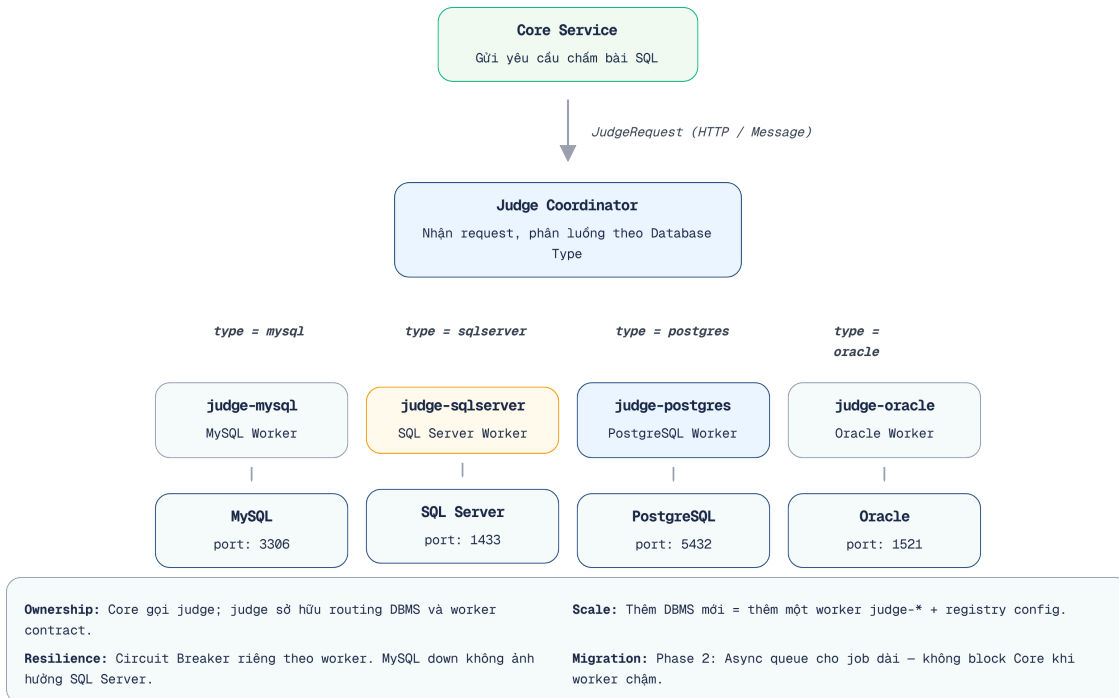
KBM hỗ trợ sinh viên thực hành SQL trên nhiều DBMS. Mỗi DBMS chạy trong sandbox container riêng biệt. Khi sinh viên nộp bài, hệ thống cần:

1. Xác định DBMS nào (dựa vào câu hỏi)
2. Gửi SQL đến đúng sandbox service
3. So sánh kết quả với đáp án (SHA-256 hash)

4. Trả về kết quả

4.5.2. Hiện trạng: Strategy Pattern + OpenFeign

Với yêu cầu hỗ trợ đa dạng các hệ quản trị cơ sở dữ liệu (DBMS), kiến trúc hiện tại của hệ thống KBM sử dụng mẫu thiết kế Strategy kết hợp cùng OpenFeign. Trong mô hình phối hợp này, một dịch vụ điều phối trung tâm mang tên **judge** được xây dựng để hoạt động như một thư ký hội đồng, chịu trách nhiệm tiếp nhận toàn bộ các yêu cầu chấm bài từ hệ thống Core. Sau đó, dựa trên loại cơ sở dữ liệu được chỉ định, nó sẽ phân tích và điều hướng (dispatch) các đoạn mã SQL này đến đúng các máy trạm xử lý độc lập (worker) chuyên trách tương ứng, đảm bảo sự phân tách trách nhiệm rõ ràng.



Hình 4.8: SQL Judge coordinator — dispatch SQL đến các worker **judge-*** theo DBMS

Core Service không nên biết chi tiết MySQL hay SQL Server được xử lý ở worker nào. Nó gửi yêu cầu chấm đến **judge**; **judge** làm nhiệm vụ coordinator: nhận database type, chọn worker tương ứng (**judge-mysql**, **judge-sqlserver**, ...), gọi worker qua HTTP/Feign hoặc queue, rồi chuẩn hóa kết quả trả về Core. Đây là pattern phổ biến: một coordinator giữ contract nghiệp vụ ổn định, các processor phía sau chuyên môn hóa theo runtime/DBMS.

4.5.3. Phân tích các vấn đề

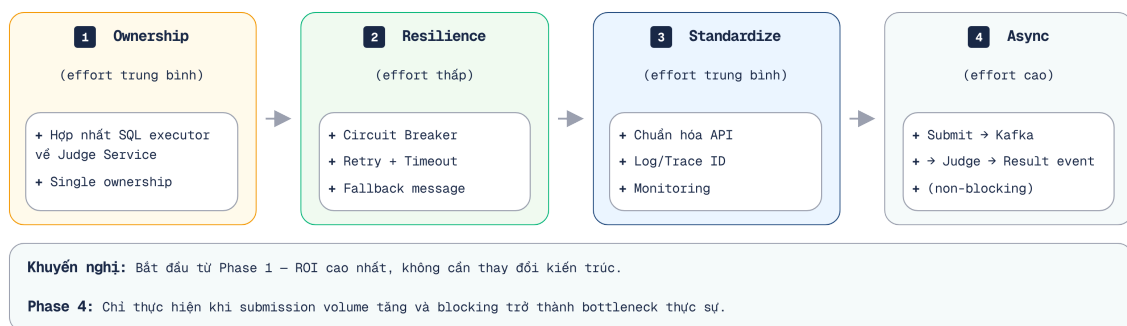
Mặc dù kiến trúc hiện hành với thiết kế Strategy Pattern đã giúp tách biệt các trình điều khiển cơ sở dữ liệu một cách trực quan, nó vẫn bộc lộ nhiều điểm nghẽn nghiêm trọng khi vận hành ở quy mô lớn hoặc khi đối mặt với các lỗi mạng không lường trước. Việc nhìn nhận rõ những hạn chế này là bước đầu tiên để tiến tới một hệ thống phân tán bền vững (resilient) và có độ tự chủ cao hơn. 4.6 dưới đây trình bày chi tiết năm vấn đề cốt lõi của thiết kế hiện tại cùng với các phương pháp giải quyết tối ưu.

#	Vấn đề	Mô tả	Best Practice
1	Routing ownership chưa rõ	Nếu Core cũng chứa logic chọn DBMS, routing bị rò rỉ ra ngoài Judge boundary	Single ownership: judge là coordinator duy nhất
2	Định tuyến cứng (hardcoded routing) dựa trên các ID bí ẩn (Magic IDs)	Database type xác định bằng hardcoded UUID/string	Enum hoặc configuration-driven registry
3	Tính chống chịu (resilience) theo từng worker chưa rõ ràng	judge-mysql ngừng hoạt động không được phép làm gián đoạn toàn bộ dịch vụ judge hay các worker khác	Circuit Breaker/Timeout riêng cho từng worker
4	Cơ chế dự phòng (fallback) chưa nhất quán	Worker của DBMS gặp sự cố → người dùng nhận lỗi kỹ thuật hoặc tiến trình nộp bài bị kẹt (stuck)	Tin nhắn dự phòng (fallback message), thử lại/hàng đợi, trạng thái rõ ràng
5	Giao ước (contract) của worker chưa được chuẩn hóa	Các judge-* dễ trả về định dạng/kết quả (format/result) khác nhau	Chuẩn hóa JudgeRequest / JudgeResult contract

Bảng 4.6: Phân tích vấn đề SQL Judge coordinator/worker

4.5.4. Đề xuất migration

Để giải quyết các rủi ro thiết kế đã phân tích và chuẩn bị nền tảng cho sự gia tăng khối lượng tải trong tương lai, hệ thống chấm điểm SQL của KBM cần một chiến lược tái cấu trúc rõ ràng. Chúng tôi đề xuất một lộ trình nâng cấp kiến trúc gồm bốn giai đoạn, bắt đầu từ việc củng cố sự ổn định cục bộ thông qua các mẫu thiết kế bảo vệ (resilience), cho đến việc thay đổi mô hình tương tác sang dạng bất đồng bộ hướng sự kiện (event-driven). Lộ trình này đảm bảo rủi ro thấp nhất trong suốt quá trình chuyển đổi trong khi mang lại lợi ích lâu dài lớn nhất về cả mặt độ tin cậy và khả năng mở rộng.



Hình 4.9: Lộ trình migration cho SQL Judge — từ resilience đến async

Phase 1 — Làm rõ ownership – (ưu tiên cao, công sức triển khai thấp): Core chỉ gọi **judge**; toàn bộ logic định tuyến (routing) hệ quản trị cơ sở dữ liệu phải được quản lý bởi **judge** coordinator. Không để logic định tuyến hệ quản trị cơ sở dữ liệu rò rỉ sang Core hoặc các service không sở hữu nghiệp vụ chấm SQL.

Phase 2 — Thêm resilience theo worker – (ưu tiên cao): `@CircuitBreaker`, `@Retry`, `@TimeLimiter` riêng cho từng `judge-*`. Khi `judge-mysql` ngừng hoạt động → hệ thống sẽ kích hoạt phản hồi dự phòng (fallback): “Sandbox MySQL đang bận, bài sẽ được chấm khi sẵn sàng”, không ảnh hưởng `judge-sqlserver`.

Phase 3 — Chuẩn hóa worker contract – (Mức độ nỗ lực: trung bình): Tất cả worker nhận cùng `JudgeRequest`, trả cùng `JudgeResult`, định tuyến dựa trên cấu hình (config-driven) thay vì gán cứng ID.

Phase 4 — Chuyên các luồng xử lý kéo dài sang mô hình bất đồng bộ – (Mức độ nỗ lực: cao, Giá trị mang lại: lớn): Submit → Kafka/queue → `judge` coordinator → `judge-*` worker → Result event → Core cập nhật. Người dùng không phải chờ đồng bộ — sẽ nhận được thông báo (notification) khi có kết quả. Đây chính là pattern mà KBM đã bắt đầu implement (Chương 5).

4.5.5. Tích hợp ngoài: QLĐT, GitHub Classroom và Network Practice

Không phải mọi tích hợp trong KBM đều nên đi qua cùng một protocol. Với QLĐT hoặc GitHub Classroom, KBM nên đặt **Anti-Corruption Layer** ở boundary: adapter dịch các giao ước (contract) bên ngoài thành lệnh/sự kiện nội bộ, che giấu định dạng cấu trúc dữ liệu, các lỗi và giới hạn tỷ lệ (rate limit) của hệ thống đối tác. Với Network Practice, điểm đáng học là ngược lại: context này được thiết kế độc lập để đánh giá giao tiếp mạng, không phụ thuộc vào service chấm SQL. Vì vậy, nó là ví dụ tốt cho bounded context theo protocol, còn vấn đề coordinator/worker cần phân tích nằm ở SQL Judge.

❗ Build vs Buy --- tự viết hay tích hợp DOMjudge cho bộ chấm code

Cùng tinh thần Anti-Corruption Layer, module chấm code đa ngôn ngữ (`kbm-judge-code`) minh họa một quyết định ‘**tự xây dựng hay mua/tích hợp**’ (**build-vs-buy**) điển hình. Thế hệ đầu là bộ chấm đồng bộ *tự viết* (C/C++ và Java) — kiểm soát tối đa nhưng nhóm phát triển phải tự đảm đương và duy trì toàn bộ trình biên dịch (compiler), môi trường thực thi (runner) cũng như các cơ chế bảo mật. Thế hệ hiện tại thay bằng **DOMjudge** — một dịch vụ OSS đã được kiểm chứng qua hàng nghìn kỳ thi ICPC. Bằng việc chấp nhận một thành phần phụ thuộc (dependency) bên ngoài, KBM loại bỏ cả một lớp rủi ro tự xây (sandbox, đa ngôn ngữ, bảo trì compiler). Bài học sync communication: với một service đồng bộ phức tạp và nhạy cảm bảo mật, *tích hợp một giải pháp đã kiểm chứng* thường thắng *tự duy trì* — miễn đặt adapter rõ ràng ở boundary để cô lập contract ngoài. (Khía cạnh cô lập/sandbox bảo mật: Ch.9.)

📖 Chuyên dịch Sync → Async

Nhiều hệ thống bắt đầu với sync (đơn giản, dễ debug) rồi chuyển sang async khi cần scale. Richardson trong [2a] minh họa tiến trình này: tiến trình thường bắt đầu với các lệnh gọi REST giữa các dịch vụ, sau đó chuyển sang mẫu thiết kế Saga khi phát sinh nhu cầu giao dịch phân tán (distributed transaction). Hệ thống KBM đang ở giữa tiến trình chuyển đổi này: một số luồng xử lý (flow) đã chuyển sang mô hình bất đồng bộ (sử dụng Kafka để nộp bài), một số luồng vẫn duy trì đồng bộ (sử dụng Feign để truy vấn), và phân hệ SQL Judge cần được tách bạch rõ ràng giữa coordinator và worker trước khi tiến hành mở rộng theo chiều ngang (horizontal scaling) một cách an toàn. Chương 5 sẽ tiếp tục phần async.

Quá trình chuyển đổi từ mô hình đồng bộ sang bất đồng bộ cần được thực hiện từng bước. Việc lạm dụng các lệnh gọi đồng bộ mà không có cơ chế bảo vệ thường dẫn đến những ảnh hưởng nghiêm trọng đối với tính sẵn sàng của hệ thống. Giống như việc một Phòng Đào tạo chỉ có một ô cửa tiếp nhận hồ sơ; nếu một sinh viên đứng đó quá lâu vì trục trặc giấy tờ (thiếu timeout/circuit breaker), toàn bộ hàng chờ phía sau sẽ bị đình trệ. Do đó, trước khi có thể chuyển đổi sang quy trình nộp hồ sơ qua hòm thư điện tử (bất đồng bộ), việc thiết lập các nguyên tắc giới hạn thời gian xử lý tại quầy là rất quan trọng.

⚠ Sai lầm thường gặp khi sử dụng Giao tiếp Đồng bộ

Khi triển khai các cuộc gọi API đồng bộ, các nhóm phát triển thường mắc phải một số lỗi kinh điển dẫn đến sự sụp đổ dây chuyền của hệ thống (cascading failure). Nhận diện và phòng tránh những cái bẫy này là nền tảng để xây dựng hệ thống bền bỉ.

Dưới đây là các sai lầm phổ biến và cách phòng tránh:

Không đặt timeout cho inter-service calls (Tin vào timeout mặc định) – Nhiều HTTP client (kể cả Feign nếu không cấu hình) có timeout mặc định rất dài hoặc chờ vô hạn. Giống như việc sinh viên đứng chờ mãi ở phòng giáo vụ mà không biết người phụ trách đã nghỉ ốm. Hậu quả: khi dịch vụ phía sau (downstream) phản hồi chậm hoặc bị treo, luồng xử lý của bên gọi bị giữ lại vô thời hạn → dẫn đến cạn kiệt bể luồng (thread pool) → gây lỗi dây chuyền (cascading failure). *Phòng tránh:* Luôn cấu hình thời gian chờ kết nối/đọc (connect/read timeout) một cách tường minh (Netflix chuẩn: 1-3 giây cho internal calls).

Tin tưởng rằng network luôn reliable – Viết code như thể mọi HTTP call đều thành công. Hậu quả: lỗi đầu tiên ở môi trường thực tế (production) mới phát hiện không có retry, không fallback. *Phòng tránh:* Áp dụng resilience patterns từ đầu (Circuit Breaker + Retry + Timeout) — ngay cả khi chưa cần scale (§4.4).

Thử lại mà không có độ trễ lùi dần (backoff) và không kiểm tra idempotency – Thử lại ngay lập tức một thao tác POST (tạo duplicate data). Giống như sinh viên bấm nút nộp bài thi liên tục khi mạng chậm, dẫn đến nộp 10 bản sao. Hậu quả: vừa khuếch đại tải lên service đang yếu (tạo bão thử lại - retry storm), dữ liệu sai, user bị tính điểm trùng. *Phòng tránh:* Chỉ retry thao tác idempotent (GET, PUT, DELETE) hoặc dùng idempotency key cho POST, dùng exponential backoff + jitter, giới hạn số lần.

Circuit Breaker không có cơ chế dự phòng (fallback) hợp lý – Mở circuit nhưng vẫn ném exception thô cho người dùng. *Phòng tránh:* Định nghĩa fallback trả về giá trị mặc định/cache hoặc thông báo “Hệ thống đang bận, sẽ xử lý sau” để đảm bảo graceful degradation.

Chuỗi gọi đồng bộ quá dài – $A \rightarrow B \rightarrow C \rightarrow D$ đồng bộ. Hậu quả: độ trễ cộng dồn, và sự chậm trễ của một mắt xích sẽ làm đình trệ toàn bộ chuỗi giao tiếp — chính là distributed monolith. Việc bắt một sinh viên phải tự chạy qua 4 phòng ban khác nhau để xin đủ 4 chữ ký trong cùng một buổi sáng là không khả thi. *Phòng tránh:* Rút ngắn chuỗi, cân nhắc async (Chương 5) cho các bước không cần kết quả ngay.

4.6. Tổng kết

Giao tiếp đồng bộ là mô hình đơn giản nhất nhưng tiềm ẩn rủi ro lớn nhất trong distributed system. Temporal coupling, cascading failures, và latency accumulation là ba vấn đề cốt lõi mà developer phải đối mặt.

OpenFeign đơn giản hóa service-to-service calls thành Java interface declarations, loại bỏ boilerplate HTTP code. Service Discovery (Eureka) giải quyết bài toán “tìm service ở đâu” — một vấn đề không tồn tại trong monolith nhưng đặc biệt quan trọng trong microservices.

Resilience patterns — Timeout, Retry, Circuit Breaker, Bulkhead — không phải optional. Chúng là điều kiện tiên quyết để hệ thống microservices hoạt động ổn định trong production. Phân tích hệ thống KBM cho thấy việc thiếu vắng các mẫu thiết kế bảo vệ (resilience patterns) là lỗ hổng (gap) nghiêm trọng nhất trong giao tiếp đồng bộ.

Ở Chương 5, chúng ta sẽ khám phá **giao tiếp bất đồng bộ** — giải pháp cho những giới hạn của giao tiếp đồng bộ (sync): Apache Kafka, kiến trúc hướng sự kiện (event-driven architecture), và cách KBM ứng dụng cơ chế truyền tin (messaging) cho luồng xử lý nộp bài (submission pipeline).

i Bài tập Chương 4

Xem Phụ lục Bài tập để luyện tập:

- 4-1** – Debugging: Cascading Failure – “Tại sao cả hệ thống ngừng hoạt động?” ([L3])
- 4-2** – REST vs gRPC vs GraphQL – Chọn gì cho scenario nào? (Trade-off Analysis [L2])

4.7. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Interprocess Communication, Service Discovery
2. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, Smart Endpoints and Dumb Pipes
3. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.7: Resilience & Reliability

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Coupling Taxonomy (design-time vs runtime), Iceberg Principle, DRY in distributed systems; Ch.9: IPC patterns
5. Michael Nygard, *Release It!*, 2nd Ed. (2018) — Circuit Breaker, Bulkhead, Timeout patterns

Nguồn trực tuyến:

- [W7] Netflix Technology Blog — “Making the Netflix API More Resilient” (2011)
- Resilience4j documentation — resilience4j.readme.io

Tài liệu tham khảo

- Richardson, C. (2018). *Microservices Patterns: With examples in Java*, 1st Edition. Manning Publications.
- Richardson, C. (2025). *Microservices Patterns: With examples in Java*, 2nd Edition. Manning Publications.